

**NAME**

`remove_orphans.py` – quarantine orphaned files in a Symphony Rptsched directory

**SYNOPSIS**

`remove_orphans.py --data-dir DIR --quarantine-dir DIR [--execute | --restore RUN_DIR]`

**DESCRIPTION**

`remove_orphans.py` finds *orphan* file groups in a Symphony ILS report-scheduler data directory (*rptsched*) and moves them into a quarantine directory.

An orphan is any group of files sharing a 4-character alphanumeric id prefix (*<id>.\**, e.g. *abcd.set*, *abcd.se-lans*) for which the *schedlist* index file in `--data-dir` contains no matching line. Every legitimate saved template has exactly one *schedlist* line for its id; files whose id is absent from *schedlist* are considered abandoned and are candidates for removal.

This tool never deletes anything: matching files are moved (not copied) into a timestamped run subfolder under `--quarantine-dir`, and a *manifest.csv* is written recording the original location of every moved file, so the run can later be reversed with `--restore`.

Unlike the stale-templates and scheduled-report categories, orphans have no corresponding *schedlist* line to begin with, so this tool never modifies *schedlist*.

**OPTIONS**

`--data-dir DIR`

Path to the *rptsched* data directory to scan. Required in every mode, including `--restore`. Never hardcoded, so the same script can be pointed at a local test copy before ever touching the production directory.

`--quarantine-dir DIR`

Path to the quarantine directory that holds run subfolders. Created if it does not already exist. Required in every mode.

`--execute`

Perform the move. Without this flag the script runs in dry-run mode: it reports what would be moved and writes nothing.

`--restore RUN_DIR`

Reverse a previous `--execute` run by moving every file recorded in *RUN\_DIR*/*manifest.csv* back to its original location. Mutually exclusive with `--execute`.

**MODES****Dry run (default)**

With neither `--execute` nor `--restore` given, the script only prints what it found: the count of orphan ids and files, followed by one line per orphan id listing its files. Nothing on disk is touched.

**Execute**

With `--execute`, the script:

1. Creates `--quarantine-dir/orphans_TIMESTAMP` (*TIMESTAMP* is *YYYYMMDD\_HHMMSS* at the moment the run starts).
2. Moves every file in every orphan group into that run subfolder.
3. Writes *manifest.csv* into the run subfolder with one row per moved file (id, filename, extension, source\_path, dest\_path, moved\_at).

If a move fails partway through, the files already moved stay moved, a partial *manifest.csv* is written for exactly those files, and the script exits non-zero. Nothing is rolled back automatically – resolve the underlying error (e.g. a permissions problem) and either re-run against the remaining orphans or use `--restore` on the partial run.

**Restore**

With **--restore** *RUN\_DIR*, the script reads *RUN\_DIR/manifest.csv* and moves each file from its recorded *dest\_path* back to its recorded *source\_path*.

Restore is idempotent and resumable: any manifest row whose *dest\_path* no longer exists is treated as already restored and silently skipped. If a row's *source\_path* already exists (e.g. a same-id file was recreated since quarantine), the script aborts immediately with a non-zero exit rather than overwriting it. Re-running **--restore** on the same *RUN\_DIR* is always safe – it simply skips whatever was already restored and resumes from where it left off. *manifest.csv* itself is never deleted, so it doubles as a permanent audit record of the run.

**EXIT STATUS**

- 0** Success (including a clean dry run, or a restore that skipped already-restored rows).
- 1** A move or restore operation failed. Details are printed to stderr.

**FILES**

*<data-dir>/schedlist*

Read to determine which ids are *not* orphaned. Never modified by this tool.

*<quarantine-dir>/orphans\_<timestamp>/manifest.csv*

Written by **--execute**; read by **--restore**. Columns: id, filename, extension, source\_path, dest\_path, moved\_at.

**EXAMPLES**

Preview what would be removed, against a local test copy:

```
remove_orphans.py --data-dir ./rptsched --quarantine-dir ./quarantine
```

Actually quarantine the orphans found above:

```
remove_orphans.py --data-dir ./rptsched --quarantine-dir ./quarantine --execute
```

Undo a specific run:

```
remove_orphans.py --data-dir ./rptsched --quarantine-dir ./quarantine --restore ./quarantine/orphans_20260826_140501
```

**SEE ALSO**

**remove\_stale\_templates.py(1)**, **sync\_operator\_field.py(1)**

Domain reference: *rptsched-domain-reference.md* in the rptsched-cleanup repository, in particular the definition of *orphan* under "Definitions".

**CAVEATS**

Requires Python 3.6.8, standard library only – there is no dependable way to install third-party packages on the production RHEL8 host this tool targets.

Quarantine is always a move, never a delete. Nothing this tool does is destructive on its own; data loss would require also deleting the quarantine directory afterward.